Database Design

Inventory Management System (IMS) for Thrifted & Branded Surplus Clothing (with Item Approval Workflow)

Prepared By: Team SQ-Way

1. Design basis

The Development Plan names three tables for the database design: Items, Users and Status Logs. This design uses exactly those three tables and takes its fields and rules from the Business Requirement.

Source	What it requires	Design response
Business Requirement 1.2 (Staff)	Add an item with brand, size, condition, price and photo; status set to Pending automatically; view own items	Items table with those fields; status defaults to Pending; added_by links an item to its Staff user
Business Requirement 1.1 and 1.2 (Reviewer)	Approve moves Pending to In Stock; reject returns the item to Pending with a mandatory reason	status and rejection_reason on Items; the reason is required by a constraint on Status Logs
Business Requirement 1.2 (System-wide)	Status Guard Rule; every status change logged with item, previous status, new status, user, date and time	Status Logs table; a constraint allows only the four valid status changes
Business Requirement 1.2 (Admin)	Search by brand, size, condition, status, price; monitor stock; generate status reports with rejection reasons and history	Indexes on the searched fields; stock counts and reports are read from Items and Status Logs
Business Requirement 1.3	RBAC; hashed passwords; search under 2 seconds; 2,000+ items	role and password_hash on Users; indexes on Items
Development Plan (Week 7)	Database design (Items, Users, Status Logs); tables created with the project setup	Section 7 SQL script creates the three tables

1.1 Design decisions

- One row in Items is one garment. The documents describe individual pieces with their own brand, size, condition, price and photo, so there is no quantity field. The stock count is the number of items with the status In Stock.
- The status is stored on the item and every change is also written to Status Logs. The item row answers “what is it now?” quickly, and the log answers “how did it get here?”.
- Rejection is not a status. A rejected item stays Pending, the latest reason is kept on the item, and every rejection is kept in the log with its reason.
- Nothing is deleted. Foreign keys restrict deletion, so the status history stays complete for the reports.
- The Status Guard Rule is enforced in the database as well as in the application, so an invalid status change is blocked even if the application has a bug.

2. Entity relationship diagram

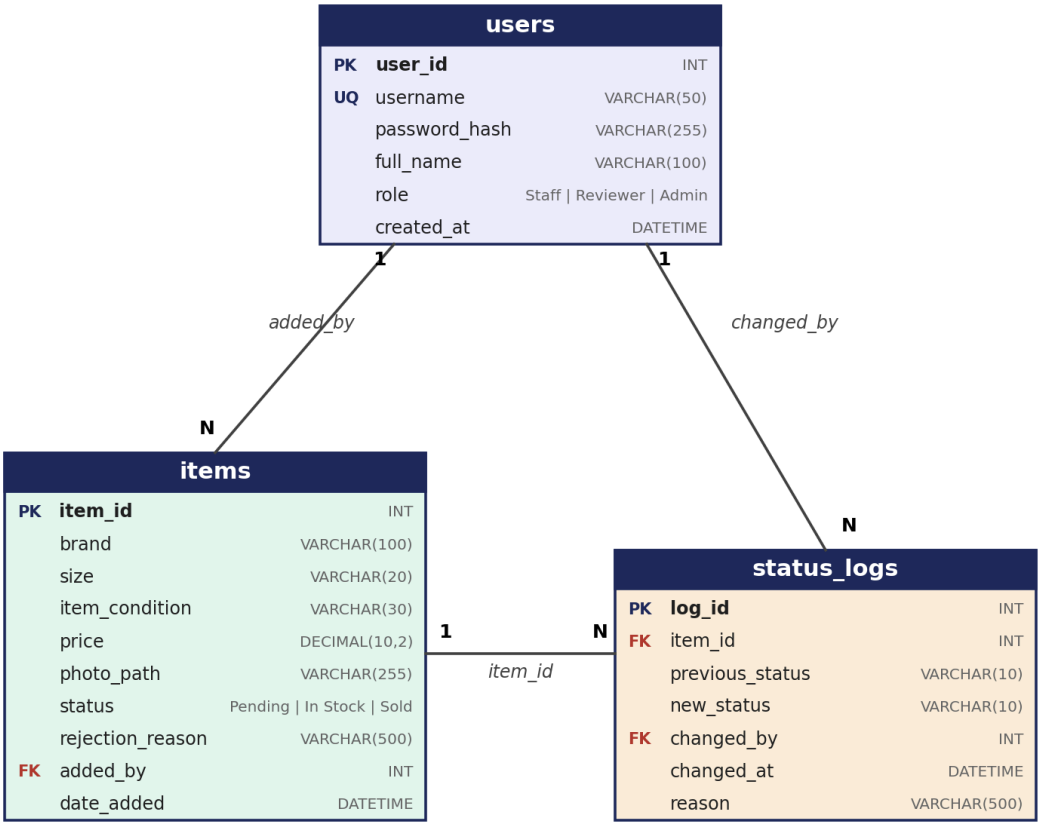


Figure 1. Entity relationship diagram: PK = primary key, FK = foreign key, UQ = unique.

Relationship	Cardinality	Meaning
users → items (added_by)	One to many	A Staff user adds many items; each item has one user who added it.
items → status_logs (item_id)	One to many	An item has many log entries, from its creation to its sale.
users → status_logs (changed_by)	One to many	A user makes many status changes; each log entry records one user.

3. Data dictionary

3.1 users

One row for each person who can log in. The role decides which functions the person can use.

Field	Type	Null	Key	Description and rule
user_id	INT	No	PK	Automatically generated ID
username	VARCHAR(50)	No	Unique	Login name
password_hash	VARCHAR(255)	No		Hashed password only; plain text is never stored (NFRS Security)
full_name	VARCHAR(100)	No		Name shown in the status history and reports
role	VARCHAR(10)	No		Staff, Reviewer or Admin (RBAC)

Field	Type	Null	Key	Description and rule
created_at	DATETIME	No		When the account was created; defaults to the current time

3.2 items

One row for each garment, from the moment Staff add it until it is sold.

Field	Type	Null	Key	Description and rule
item_id	INT	No	PK	Automatically generated ID
brand	VARCHAR(100)	No		Brand of the garment
size	VARCHAR(20)	No		Size of the garment
item_condition	VARCHAR(30)	No		Condition entered by Staff. Named item_condition because “condition” is a reserved word in some databases.
price	DECIMAL(10,2)	No		Selling price; must be greater than 0
photo_path	VARCHAR(255)	No		Location of the uploaded photo file. The image itself is stored as a file, not in the table.
status	VARCHAR(10)	No		Pending, In Stock or Sold. Defaults to Pending.
rejection_reason	VARCHAR(500)	Yes		Latest rejection reason. Only allowed while the status is Pending and cleared when the item is approved.
added_by	INT	No	FK	The Staff user who added the item (users.user_id)
date_added	DATETIME	No		When the item was added; defaults to the current time

3.3 status_logs

One row for every status change. Rows are only added, never edited or deleted.

Field	Type	Null	Key	Description and rule
log_id	INT	No	PK	Automatically generated ID
item_id	INT	No	FK	The item that changed (items.item_id)
previous_status	VARCHAR(10)	Yes		Status before the change; empty when the item is first added
new_status	VARCHAR(10)	No		Status after the change
changed_by	INT	No	FK	The user who made the change (users.user_id)
changed_at	DATETIME	No		Date and time of the change; defaults to the current time
reason	VARCHAR(500)	Yes		Rejection reason; required for a rejection and empty for every other change

4. Indexes

Index	Table and columns	Used for
idx_items_status	items (status)	Pending Items queue, stock counts, status reports, search by status
idx_items_brand_size	items (brand, size)	Search by brand and size; low-stock grouping
idx_items_price	items (price)	Search by price
idx_logs_item_time	status_logs (item_id, changed_at)	Status history of one item, in time order
idx_logs_changed_at	status_logs (changed_at)	Reports over a date range

The foreign key columns (added_by, item_id, changed_by) are indexed automatically by MySQL. In PostgreSQL they need their own indexes.

5. How the use cases use the database

Use case	Database operation
UC-01 Log in	Read users by username; compare the password hash; read role
UC-02 Add new item	Insert into items (status Pending); insert into status_logs (T1)
UC-03 View my items and status	Read items where added_by is the current user
UC-04 View Pending Items queue	Read items where status is Pending
UC-05 Approve item	Update items to In Stock and clear rejection_reason; insert into status_logs (T2)
UC-06 Reject item with reason	Update items.rejection_reason; insert into status_logs (T3) with the reason
UC-07 Process sale	Update items to Sold where status is In Stock; insert into status_logs (T4)
UC-08 Search inventory	Read items filtered by brand, size, condition, status and price
UC-09 Monitor stock levels and alerts	Count items by status; group In Stock items by brand and size
UC-10 Generate status reports	Join items, status_logs and users for the chosen status
SYS-01 Log every status change	Insert into status_logs inside the same transaction as the item update
SYS-02 Enforce the Status Guard Rule	Constraints on items and status_logs reject invalid values and changes

6. Key queries

Values written as :name are parameters filled in by the application.

6.1 Pending Items queue (UC-04)

```
SELECT item_id, brand, size, item_condition, price, photo_path,
       rejection_reason, date_added
FROM items
WHERE status = 'Pending'
ORDER BY date_added;
```

6.2 Approve an item (UC-05)

```
START TRANSACTION;
UPDATE items SET status = 'In Stock', rejection_reason = NULL
WHERE item_id = :item_id AND status = 'Pending';
-- the application continues only if exactly one row was updated
INSERT INTO status_logs (item_id, previous_status, new_status, changed_by)
VALUES (:item_id, 'Pending', 'In Stock', :reviewer_id);
COMMIT;
```

6.3 Reject an item (UC-06)

```
START TRANSACTION;
UPDATE items SET rejection_reason = :reason
WHERE item_id = :item_id AND status = 'Pending';
INSERT INTO status_logs (item_id, previous_status, new_status, changed_by, reason)
VALUES (:item_id, 'Pending', 'Pending', :reviewer_id, :reason);
COMMIT;
```

6.4 Process a sale (UC-07)

```
START TRANSACTION;
UPDATE items SET status = 'Sold'
WHERE item_id = :item_id AND status = 'In Stock';
-- zero rows updated means the item was Pending or Sold: the sale is blocked
INSERT INTO status_logs (item_id, previous_status, new_status, changed_by)
VALUES (:item_id, 'In Stock', 'Sold', :staff_id);
COMMIT;
```

6.5 Search inventory (UC-08)

```
SELECT item_id, brand, size, item_condition, price, status
FROM items
WHERE (:brand  IS NULL OR brand = :brand)
  AND (:size   IS NULL OR size = :size)
  AND (:cond   IS NULL OR item_condition = :cond)
  AND (:status IS NULL OR status = :status)
  AND (:min_price IS NULL OR price >= :min_price)
  AND (:max_price IS NULL OR price <= :max_price)
ORDER BY brand, size;
```

6.6 Stock levels and low-stock alerts (UC-09)

```
-- Count of items per status
SELECT status, COUNT(*) AS total FROM items GROUP BY status;

-- Brand and size groups with fewer than :threshold items In Stock
SELECT brand, size, SUM(status = 'In Stock') AS in_stock
FROM items
GROUP BY brand, size
HAVING SUM(status = 'In Stock') < :threshold;
```

The low-stock threshold is not defined in the Business Requirement or the Development Plan, so it is a setting the team chooses.

6.7 Status report with history (UC-10)

```
SELECT i.item_id, i.brand, i.size, i.status,
       l.previous_status, l.new_status, l.reason,
       u.full_name AS changed_by, l.changed_at
FROM items i
JOIN status_logs l ON l.item_id = i.item_id
JOIN users u       ON u.user_id = l.changed_by
WHERE i.status = :status
ORDER BY i.item_id, l.changed_at;
```

7. SQL script

The full script is also supplied as the file ims_schema.sql so the team can run it during project setup (task IMS-03-01-01).

```
-- IMS Database Schema (Items, Users, Status Logs)
-- Team SQ-Way | Task IMS-02-02-01 Database Design
-- Written for MySQL 8.0.16+ (CHECK constraints are enforced). Adapt types if another engine is used.

CREATE TABLE users (
  user_id       INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  username      VARCHAR(50)  NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,            -- hashed value only, never plain text
  full_name     VARCHAR(100) NOT NULL,
  role          VARCHAR(10)  NOT NULL,
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_users_role CHECK (role IN ('Staff', 'Reviewer', 'Admin'))
);

CREATE TABLE items (
  item_id          INT           NOT NULL AUTO_INCREMENT PRIMARY KEY,
  brand            VARCHAR(100)  NOT NULL,
  size             VARCHAR(20)   NOT NULL,
  item_condition   VARCHAR(30)   NOT NULL,
  price            DECIMAL(10,2) NOT NULL,
  photo_path       VARCHAR(255)  NOT NULL,
  status           VARCHAR(10)   NOT NULL DEFAULT 'Pending',
  rejection_reason VARCHAR(500)  NULL,            -- latest rejection reason, empty once approved
  added_by         INT           NOT NULL,
  date_added       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_items_price  CHECK (price > 0),
  CONSTRAINT chk_items_status CHECK (status IN ('Pending', 'In Stock', 'Sold')),
  CONSTRAINT chk_items_reason CHECK (rejection_reason IS NULL OR status = 'Pending'),
  CONSTRAINT fk_items_user    FOREIGN KEY (added_by) REFERENCES users (user_id) ON DELETE RESTRICT
);

CREATE TABLE status_logs (
  log_id          INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
  item_id         INT          NOT NULL,
  previous_status VARCHAR(10)  NULL,              -- NULL when the item is first added
  new_status      VARCHAR(10)  NOT NULL,
  changed_by      INT          NOT NULL,
  changed_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reason          VARCHAR(500) NULL,              -- filled only for a rejection
  CONSTRAINT fk_logs_item FOREIGN KEY (item_id)    REFERENCES items (item_id) ON DELETE RESTRICT,
  CONSTRAINT fk_logs_user FOREIGN KEY (changed_by) REFERENCES users (user_id) ON DELETE RESTRICT,
  -- Status Guard Rule: only these four changes can ever be logged
  CONSTRAINT chk_logs_transition CHECK (
      (previous_status IS NULL      AND new_status = 'Pending'  AND reason IS NULL)          -- T1 new item
   OR (previous_status = 'Pending'  AND new_status = 'In Stock' AND reason IS NULL)          -- T2 approve
   OR (previous_status = 'Pending'  AND new_status = 'Pending'  AND reason IS NOT NULL
                                    AND CHAR_LENGTH(TRIM(reason)) > 0)                      -- T3 reject
   OR (previous_status = 'In Stock' AND new_status = 'Sold'     AND reason IS NULL)          -- T4 sale
  )
);

-- Indexes for search (under 2 seconds with 2,000+ items), the Pending queue, and reports
CREATE INDEX idx_items_status      ON items (status);
CREATE INDEX idx_items_brand_size  ON items (brand, size);
CREATE INDEX idx_items_price       ON items (price);
CREATE INDEX idx_logs_item_time    ON status_logs (item_id, changed_at);
CREATE INDEX idx_logs_changed_at   ON status_logs (changed_at);
-- Foreign key columns (added_by, item_id, changed_by) are indexed automatically by MySQL InnoDB.
```